Test Strategy

Overview

This homwork verifies the MediTrack application using unit, integration, and system testing. The goal was to test individual functions, interactions between modules, and the complete clinic workflow while following the HW3 requirements.

Unit Testing

Unit tests were written for the Billing and Records modules. Billing tests verify invoice creation and insurance processing using valid inputs, invalid inputs, boundary values, and error conditions.
Records tests verify diagnosis validation, inactive patients, missing appointments, incorrect patient IDs, invalid appointment states, and updating existing records. Each unit test focuses on one function at a time.

Stubs Used

For unit testing, database access was isolated using Python's unittest.mock library (MagicMock and patch). Mock database connections and cursors were used so that only the logic inside the function under test was verified. No mocks or stubs were used during integration or system testing because those tests used the provided fresh_db fixture and a real SQLite test database.

Integration Scope

Integration tests verified that different MediTrack modules worked correctly together. Scenarios included patient registration, appointment scheduling, deactivating patients, cancelling appointments, duplicate patient registration, invoice creation, and insurance processing. These tests confirmed that data was passed correctly between modules and stored in the database.

System Test Rationale

A complete end-to-end clinic visit was tested using the real database. The workflow included registering a patient, scheduling an appointment, checking in the patient, creating a diagnosis and prescription, completing the visit, generating an invoice, and processing payment. Assertions were made after each major step to verify returned values, application state, and database records. This test demonstrates that the full application workflow operates correctly from beginning to end.

Coverage Analysis

Coverage.py was used to measure statement and branch coverage. Additional tests were added to improve coverage until the assignment targets were met. The final results satisfied the required statement coverage for patients.py, appointments.py, records.py, and billing.py.

The final statement coverage results were:

- Patients module: **91%**
- Appointments module: **83%**
- Records module: **86%**
- Billing module: **84%**

Conclusion

Using unit, integration, and system testing provided confidence that individual functions, module interactions, and the complete MediTrack workflow behaved correctly. The combination of mocked unit tests and real database integration/system tests resulted in a reliable and well-tested application.